

ROADMAP DE IMPLEMENTAÇÃO

AETHERFORGE

Caçadores da Fratura

Action-loot RPG de navegador — caça, loot com raridade, e a **Forja Ativa**: refino de itens (+0 → +11) que mistura sorte e habilidade.

- Comum
- Incomum
- Raro
- Épico
- Lendário
- Mítico
- Divino

Gerado em 2026-07-12 · 9 fases · 56 features rastreadas

ROADMAP — Aetherforge

Roadmap de implementação por fases. Este documento é a **fonte do PDF** ([dist/Aetherforge-Roadmap.pdf](#)). O status vivo de cada feature está em [FEATURES.md](#) e deve ser atualizado a cada implementação.

Legenda de status: ☐ Não iniciado · ☒ Em andamento · ☒ Concluído

Visão de entregas

Fase	Nome	Objetivo	Entregável
0	Fundações	Repo, stack, tokens, CI	Projeto roda, testes verdes
1	Núcleo de Itens	Geração de itens + raridade	core/ gera itens data-driven
2	Inventário & Equipamento	Guardar, equipar, comparar	Tela de inventário funcional
3	Refino & Forja Ativa	Evolução +0..+11	Forja com mini-jogo jogável
4	Combate & Fraturas	Caçar e dropar	1 Fratura jogável com drops
5	Progressão & Economia	Níveis, recursos, sumidouros	Loop completo fechado
6	Polish Visual	UI moderna, "juicy", responsiva	MVP apresentável
7	Persistência & Meta	Save, contas, leaderboard	Backend + meta-progressão
8	Conteúdo & Live-ops	Mais Fraturas, eventos, balanço	Jogo em evolução contínua

MVP jogável = Fases 0-6.

Fase 0 — Fundações ☒

Meta: base técnica sólida e reproduzível. **Concluída em 2026-07-12.**

- ☒ Scaffold Vite + React + TypeScript
- ☒ Tailwind (v4) + design tokens (cores de raridade, tipografia, vidro) — ver [design/11](#)
- ☒ Estrutura de pastas [core/](#) [state/](#) [ui/](#) [data/](#) — ver [design/12](#)
- ☒ RNG semeado ([core/rng.ts](#)) + testes
- ☒ Loader + validação (zod) dos JSON de [data/](#)
- ☒ Vitest configurado; ESLint + Prettier
- ☒ CI (lint + typecheck + testes + build no push)
- ☒ _Extra:_ seam de cena Phaser ([ui/combat/PhaserMount.tsx](#)) para a ação em cena futura

Fase 1 — Núcleo de Itens ☒

Meta: gerar um item completo a partir de dados. **Concluída em 2026-07-12.**

- ☒ Tipos TS do modelo de dados ([design/13](#)) — [core/items/types.ts](#)
- ☒ Sorteio de raridade por peso — [core/loot/drop.ts](#) (+ hooks de modificador p/ Fase 4/9)
- ☒ Geração de item: base → itemLevel → rolagem de atributos base — [core/items/generate.ts](#)
- ☒ Sistema de afixos (prefixos/sufixos por raridade) — [core/items/affixes.ts](#)
- ☒ Pipeline de cálculo de stats (base × raridade + afixos) — [core/items/stats.ts](#)

- ☒ Nome derivado (base + afixos + "+N") — `formatItemName`
- ☒ Testes de distribuição (100k drops batem a curva $\pm 1pp$) — `core/loot/drop.test.ts`
- ☒ _Extra:_ geração de itens integrada ao App shell (demo visual ao vivo)

Fase 2 — Inventário & Equipamento ☒

Meta: o jogador vê, equipa e gerencia itens. **Concluída em 2026-07-12.**

- ☒ Store de inventário/equipamento (Zustand) — `state/inventoryStore.ts`
- ☒ Regras de equipar (slot + nívelRequerido) — `core/equipment/rules.ts`
- ☒ Cálculo de stats do personagem (soma dos equipados) — `core/equipment/stats.ts`
- ☒ Gear Score — `core/items/gearScore.ts`
- ☒ UI: grid de inventário + comparação de delta — `ui/inventory/InventoryScreen.tsx`, `ui/components/ItemCard.tsx` (reutilizado da Fase 1)
- ☒ Ações: trancar, descartar, fusão em recursos (`core/items/fuse.ts`), filtros (`core/items/filter.ts`)
- ☒ _Extra:_ `EquipmentPanel` (9 slots) integrado ao App shell com demo ao vivo

Fase 3 — Refino & Forja Ativa ☐

Meta: o diferencial autoral funcionando.

- ☐ Config de refino carregada de `data/refinement.json`
- ☐ Cálculo de chance final (base + pedras c/ retorno decrescente + forja)
- ☐ Resolução de refino (sucesso / -1 / destruição) com Selo
- ☐ Bônus de stats por nível de refino aplicado ao item
- ☐ **Forja Ativa:** mini-jogo de precisão (barra/zona quente)
- ☐ Modo auto-forja (acessibilidade)
- ☐ UI da Forja: seleção de pedras/selo, chance, resultado dramático
- ☐ Log/histórico de tentativas

Fase 4 — Combate & Fraturas ☐

Meta: caçar mobs e dropar loot de verdade.

- ☐ Máquina de estados de combate (`core/combat/engine`)
- ☐ Cálculo de dano (com timed strike)
- ☐ Mobs e loot tables data-driven
- ☐ Drop completo: raridade (curva por tier + Sorte) → item
- ☐ 1 Fratura jogável (ondas + chefe + recompensas)
- ☐ **Ritmo de Combate:** timed strikes + skills
- ☐ Tela de combate (React + efeitos PixiJS) + reveal de loot

Fase 5 — Progressão & Economia ☐

Meta: fechar o loop e sustentar a economia.

- ☐ XP e nível do Forjador + curva
- ☐ Pontos de atributo distribuíveis + respec
- ☐ Recursos e sumidouros (ouro, pedras, selos, essência, fragmentos)
- ☐ Craft/upgrade de pedras; re-roll de afixos (Essência)
- ☐ Tiers de Fratura + gates de progressão
- ☐ Fratura do dia

Fase 6 — Polish Visual ☐

Meta: transformar o MVP em algo bonito e "juicy".

- ☐ Design system completo (componentes + Storybook)
- ☐ Animações Framer Motion (reveals, hover, level up)
- ☐ Momentos juicy (drop raro, sucesso/falha de refino)
- ☐ Responsividade mobile + acessibilidade (reduce-motion, contraste, teclado/toque)
- ☐ Som e feedback tátil

Fase 7 — Persistência & Meta ☐

Meta: durabilidade e endgame social.

- ☐ Save serializado + migrations (localStorage)
- ☐ Backend (Fastify + Postgres + Prisma) + contas
- ☐ Validação de regras no servidor (anti-cheat)
- ☐ Leaderboard (gear score / maior tier limpo)
- ☐ Compêndio/coleção

Fase 8 — Conteúdo & Live-ops ☐

Meta: manter o jogo vivo.

- ☐ Novas Fraturas, biomas, chefes
- ☐ Novas bases de item / afixos / passivos
- ☐ Eventos temporários (forja abençoada, fratura corrompida global)
- ☐ Cap de refino estendido (+13/+15) via item especial
- ☐ Telemetria e balanceamento contínuo

Riscos e mitigação

Risco	Mitigação
Balanceamento do drop/refino	Data-driven + testes de distribuição desde a Fase 1
Escopo inflar	MVP = Fases 0-6; resto é incremental
RNG frustrante	Forja Ativa dá agência; Selo evita destruição; log transparente
Performance de inventário	Virtualização de listas na Fase 2/6
Combate ficar raso	Ritmo de Combate (timed strikes) + skills por arma

Como este roadmap evolui

1. Trabalha-se por fase, de cima para baixo.
2. Ao concluir uma feature, marca-se o item em `FEATURES.md` com data e commit.
3. Ao concluir uma fase, atualiza-se o status da fase aqui e regenera-se o PDF (`node scripts/build-pdf.mjs`).